

Submission #6: Algorithmic Organizer

Prepared by

Ivan Hromovyi

in

IKT203

Algorithms and Data Structures

Department of Information and Communication Technology

Faculty of Engineering and Science

September 27, 2026

Contents

1 Report **1**

1.1 Selection and Bubble Algorithms 1

1.2 Divide and Conquer Algorithms 1

1.3 Binary vs Linear Search 1

List of Tables

1 Sorting Algorithm Performance on 5000 Bank Accounts (sorted by balance) 1
2 Search Comparisons: Linear vs. Binary Search (n = 5000, target: ACC3955) 2

1 Report

Initially, the assignment was focused on implementing different sorting algorithms where the main goal was to compare, understand and implement them. Objects as `TBankAccount` and `TLinkedList` were taken from the previous submission # 4 (Simple Search).

1.1 Selection and Bubble Algorithms

First two sorting algorithms in this assignment were `Selection Sort Algorithm` and `Bubble Sort Algorithm`. The idea behind `Selection Algorithm` is to find the smallest element in the unsorted part and put it at the beginning. Meanwhile, the `Bubble Algorithm` compares neighbouring elements and swap them if they are in the wrong order. Both algorithms have $O(n^2)$, which is not that efficient compared to more advanced complex solutions.

`Selection Algorithm` was implemented for both an array and a list. However, it is harder to implement on a linked list rather than on array mainly because arrays allow direct access by index, while linked lists require traversing nodes. Despite this added implementation complexity, the measured runtimes for the array (0.059148s) and list (0.059752s) versions were nearly identical in this run.

1.2 Divide and Conquer Algorithms

The next two sorting algorithms are `Quick Sort` for an array and `Merge Sort` for a list. Both of these solutions have a complexity of $n \log n$ and are far more efficient than the previous algorithms. The main idea behind those algorithms is to split the array or list into smaller parts, complete sorting within those parts and merge them back together. This is called the `Divide and Conquer` principle.

The following table clearly demonstrates the amount of comparisons, swaps, and time taken to complete the sorting based on `accountBalance`.

Algorithm	**Comparisons**	**Swaps**	**Time (s)**	**Complexity**
Selection Sort (Array)	12,497,500	4,988	0.059148	$O(n^2)$
Selection Sort (List)	12,497,500	5,000	0.059752	$O(n^2)$
Bubble Sort (Array)	12,497,500	6,229,105	0.113954	$O(n^2)$
Quick Sort (Array)	67,767	32,548	0.000716	$O(n \log n)$
Merge Sort (List)	55,265	55,265	0.001008	$O(n \log n)$

Table 1: Sorting Algorithm Performance on 5000 Bank Accounts (sorted by balance)

1.3 Binary vs Linear Search

The last exercise was to implement an array binary search. The main difference between linear and binary search is that the first solution checks the elements one after another,

while the second algorithm is used on already sorted data structures, where the search begins at the middle of the array and after each comparison, half of the remaining elements are discarded. The complexity of the `Linear Search` is $O(n)$, and the complexity of the `Binary Search` - $O(\log n)$. The search comparisons are shown at the Table 2.

Search Method	**Comparisons**	**Time (s)**	**Complexity**
Linear Search	101	0.000006	$O(n)$
Binary Search	13	0.000001	$O(\log n)$

Table 2: Search Comparisons: Linear vs. Binary Search (n = 5000, target: ACC3955)

The comparison counts make this clear: the three $O(n^2)$ algorithms each perform exactly $12,497,500$ comparisons regardless of implementation, while `Divide and Conquer Algorithms` require only 55.000–68.000 - which is reflected directly in clock time. This way, the report demonstrates the difference in efficiency of different popular algorithms, and proves that the sorting algorithms are powerful tools that are being used everywhere, and are essential to understand.